frevia monorepo — pnpm workspaces + Turborepo

apps

apps/web
Next.js 16 presentation layer

app/
App Router · route groups per actor · BFF route handlers

- **(public) / guest** — jobs · forum · experts
- **(client)** — client dashboard
- **(freelancer)** — freelancer dashboard
- **admin** — admin dashboard

«use» **components/** — React UI components of the screens

«use» **apiRequests/** — typed API caller modules (auth, jobs, contracts, ...)

«use» **lib/** — http client (JWT attach & refresh), auth session, utils

«use» **i18n/** — messages/ en & vi translation catalogues

apps/api
NestJS 11 application tier (REST + Socket.io)

src/routes/
33 domain modules · Controller → Service → Repository (+ DTO) · modules never import each other

- **auth**
- **proposals**
- **contract**
- **payments**
- **forums**
- **conversations**
- **disputes**
- *… +26 more modules*

«use» **src/shared/**
cross-cutting infrastructure

- guards
- filters
- interceptors
- services
- prisma client

«use» **prisma/**
schema.prisma (72 models)
migrations · seed

«use» «use» «use» «extends» «use»

packages

packages/ui
@repo/ui · shared React components · imported by apps/web only

- buttons
- dialogs
- tables
- form controls

packages/shared-type
@shared/types · single source of truth for the API contract

- model/* (Zod request / response)
- enumerations · constants
- permission names
- error codes

packages/typescript-config
@repo/typescript-config
strict TypeScript baselines extended by the applications

packages/eslint-config
@repo/eslint-config
one lint ruleset for every workspace of the monorepo

Legend

- «use» — the source imports / depends on the target
- «extends» — the source extends the configuration of the target

Rules

1. Dependencies point downwards: the two applications depend on the packages, and a package never imports from an application.
2. No route module of apps/api imports another route module; cross-module data flows through the Prisma schema.
3. Arrows drawn from the apps group apply to both apps/web and apps/api. eslint-config is applied to every workspace (the arrows from packages/ui and packages/shared-type are omitted for readability).
4. Python AI microservices (id-verification, cv-matching, ai-moderation backed by Ollama) reside in ai-services/ and communicate with apps/api via HTTP REST, independent of the TypeScript pnpm workspaces.